

The AI-Native Organization: Agentic Organizational Memory as the Operating System for Closed-Loop Enterprise Operations

Shaurya Singh
Founding Engineer, Agilow
agilow.ai
shaurya@agilow.ai

Abstract—Early-stage technical teams generate organizational knowledge continuously across meetings, planning documents, retrospectives, and asynchronous communication channels. In practice, this knowledge rarely travels between systems without a person manually carrying it. The result is a category of largely invisible labor, which this paper terms organizational reconciliation work, consumed by reconstructing context, transferring decisions across tool boundaries, and re-deriving institutional knowledge that was never durably captured. This paper argues that eliminating reconciliation work requires more than adopting AI-powered point solutions. It requires a purpose-built agentic architecture in which a centralized organizational memory layer serves as the shared substrate over which a graph-orchestrated multi-agent system executes closed-loop operational workflows. We propose a four-layer architecture comprising a data ingestion layer, an organizational memory layer with hybrid dense and graph-based retrieval, an AI reasoning layer implemented as a LangGraph stateful agent graph, and a closed-loop operations layer covering meeting-to-sprint automation, standup synthesis, retrospective-to-planning pipelines, blocker tracking, and knowledge retrieval. We ground this framework in a real deployment context at Agilow, a six-person technical delivery startup founded by a Carnegie Mellon University Tepper School of Business graduate, where Sprint Health automation, a two-phase LLM report generation pipeline, and a Slack standup bot tied to weighted sprint criteria have been built and shipped. We define three research questions concerning reconciliation work reduction, decision latency, and the moderating factors that determine which workflows are amenable to closure. We discuss the technical architecture in detail including agent roles, state schemas, checkpointing, retrieval design, and human-in-the-loop interrupt nodes. We conclude with an honest account of implementation challenges, limitations including extraction error accumulation and low signal-to-noise in chat data, and directions for future empirical and systems research.

Index Terms—organizational memory, agentic AI, LangGraph, multi-agent systems, GraphRAG, knowledge management, closed-loop workflows, enterprise automation, retrieval-augmented generation

I. INTRODUCTION

Consider a six-person team running a software delivery operation. They plan sprints in a Google Doc with a new tab for each week. They hold retrospectives in a separate Google Sheets file. Their meetings are captured by Granola, with notes sent to a WhatsApp group after each call. Daily standups happen through a Slack bot that prompts each team member in direct messages. Scheduling lives in Google Calendar.

The actual conversations that drive most decisions happen in WhatsApp threads that scroll past within hours.

None of these systems talk to each other.

When a decision gets made in a Monday Google Meet call, it shows up in Granola. The scrum master sends those notes to the WhatsApp group. Whether that decision makes it into the sprint doc, gets turned into a task, influences the next retro, or is ever findable again depends entirely on whether a person remembers to carry it there. When a blocker gets mentioned in Tuesday's standup, there is no mechanism ensuring it surfaces again on Thursday if it has not been resolved. When a new team member needs to understand why a particular architectural approach was chosen three months ago, the answer exists only in someone's memory or buried in a WhatsApp thread that nobody can efficiently search.

This is not a dysfunction specific to any one team. It is a structural property of how early-stage technical organizations operate. The modern SaaS tool stack is optimized for capturing information within individual systems and essentially ignores the problem of information traveling between them. The result is that human workers spend a measurable and largely unacknowledged proportion of their time performing what this paper calls organizational reconciliation work: reconstructing context, transferring decisions across tool boundaries, re-deriving knowledge that was captured somewhere but not made retrievable, and re-litigating questions whose answers exist in a system nobody thought to check.

Recent advances in large language model agents create, for the first time, a technically credible path to addressing this problem structurally rather than through point solutions. Systems capable of tool use, multi-step planning, stateful graph execution, and hybrid retrieval over structured and unstructured data can, in principle, automate the information transfer steps that currently require human intermediaries [14], [15]. But deploying these capabilities effectively in an organizational context requires more than plugging in an AI meeting summarizer or a chatbot. It requires a deliberate architectural decision: building a centralized organizational memory layer and running a purpose-designed multi-agent system over it.

This paper makes the following contributions:

First, we introduce and formally define the concept of organizational reconciliation work, distinguishing it from general

coordination overhead and situating it as a direct consequence of information fragmentation across heterogeneous tool stacks.

Second, we propose the AI-native organization as an organizational form defined by three structural properties: centralized organizational memory, AI-mediated knowledge operations, and closed-loop operational workflows, and distinguish this from organizations that have merely adopted AI tools.

Third, we present a concrete four-layer agentic architecture in which a LangGraph-orchestrated multi-agent system operates over a hybrid organizational memory layer to execute closed-loop workflows, describing agent roles, state schemas, retrieval design, checkpointing, and human-in-the-loop interrupt patterns with sufficient specificity to be implementable.

Fourth, we ground the framework in a real deployment at Agilow, describing what has been built, what has been learned from building it, and where the architecture is still evolving.

II. BACKGROUND AND MOTIVATION

A. Organizational Memory and Its Failure Modes

The study of organizational memory has a well-developed theoretical foundation. Walsh and Ungson [1] established the foundational account, identifying five storage bins through which organizations retain knowledge: individuals, culture, transformations, structures, and ecology, with external archives as a sixth. Their central observation, that organizational memory is inherently distributed, has become more rather than less true in the decades since. Where a 1990s organization might distribute knowledge across individuals, filing cabinets, and a shared drive, a contemporary organization distributes it across dozens of specialized SaaS platforms, each with its own data model, access control system, and retrieval interface.

Stein and Zwass [2] operationalized this framework in an information systems context, proposing a functional model of organizational memory information systems and identifying the gap between information capture and information retrieval as the central design challenge. Their observation that most organizational systems optimize for storage rather than retrieval remains accurate today. Slack captures every message. Granola captures every meeting. Google Docs captures every draft. The question of whether any of this captured information is practically accessible at the moment it is needed receives far less engineering attention.

Argote and Miron-Spektor [3] synthesized decades of organizational learning research, emphasizing that knowledge in organizations exists not just in individuals but in routines, tools, and the relationships between them. They noted that transactive memory systems, first theorized by Wegner [4], allow teams to distribute knowledge across members in ways that exceed what any individual could hold, provided the team maintains accurate meta-knowledge about who knows what. This meta-knowledge is precisely what breaks down as teams grow, tools multiply, and personnel turn over.

The costs of this breakdown are real and measurable. Employees report spending between 20 and 30 percent of working time searching for information they believe exists somewhere in their organization [10]. The cost of employee turnover,

a significant component of which is the loss of institutional knowledge that was never externalized, is estimated at between 50 and 200 percent of annual salary per departing employee [11]. These figures predate the current era of SaaS proliferation and are likely conservative by current standards.

B. The Agentic AI Turn

The technical landscape relevant to this paper has shifted dramatically between 2022 and 2025. Large language models capable of instruction following, tool use, and multi-step reasoning have moved from research curiosities to production infrastructure. Three specific technical developments are directly relevant to the architecture we propose.

The first is retrieval-augmented generation [13], which demonstrated that language models can be grounded in external knowledge stores, dramatically reducing hallucination and enabling responses that reflect current rather than training-time knowledge. Subsequent work on agentic RAG [17] extended this by allowing agents to dynamically manage retrieval strategies, iteratively refine queries, and adapt retrieval behavior based on intermediate results rather than treating retrieval as a single fixed operation.

The second is the ReAct framework [14], which established the pattern of interleaving reasoning traces with actions in language model agents. Rather than generating a response in a single forward pass, a ReAct agent thinks about what it needs, decides on a tool call, observes the result, updates its reasoning, and repeats. This pattern is fundamental to the agent design we describe in Section VII, where each specialist agent operates as a ReAct loop with access to a defined set of tools.

The third is LangGraph [18], which provides a framework for implementing multi-agent systems as stateful directed graphs where nodes are agent functions, edges are transitions, and state is a typed schema that persists across the entire graph execution via checkpointing. LangGraph’s checkpointing mechanism, which saves a complete snapshot of graph state after every node execution, is what makes long-running organizational workflows fault-tolerant and resumable. Its human-in-the-loop interrupt pattern, which pauses graph execution at defined nodes to wait for human input, is what makes automation safe at decision points that require judgment.

Microsoft’s GraphRAG [16] introduced a knowledge graph-based retrieval approach that extracts entities and relationships from text corpora, constructs community hierarchies using graph clustering algorithms, and generates summaries that support both local and global retrieval queries. For organizational memory specifically, graph-based retrieval has significant advantages over pure vector similarity search: it can answer multi-hop questions that require traversing relationships between entities, surface historical context about decisions by following provenance edges, and detect conflicts between current proposals and prior commitments.

III. RELATED WORK

Our work draws on four bodies of literature and attempts to connect them in a way that none individually addresses.

Organizational Memory and Knowledge Management Systems. Walsh and Ungson [1] and Stein and Zwass [2] provide the theoretical and systems foundations for organizational memory research. Alavi and Leidner [8] conducted an influential review of knowledge management systems, noting the persistent gap between knowledge capture and knowledge use in enterprise information systems. Davenport and Prusak [7] examined the practical barriers to organizational knowledge transfer, many of which remain relevant to AI-mediated systems. Nonaka [5] and Nonaka and Takeuchi [6] established the distinction between tacit and explicit knowledge and the challenge of externalization, which our ingestion pipeline addresses at a technical level. More recent work by Sen, Anan, and Okun [9] confirmed through qualitative study that organizational memory still primarily resides in individuals even in large, structured organizations, reinforcing the architectural motivation for our approach.

Retrieval-Augmented Generation and GraphRAG. Lewis et al. [13] introduced retrieval-augmented generation as a mechanism for grounding language model outputs in external knowledge. Edge et al. [16] extended this through GraphRAG, combining knowledge graph construction with community-based hierarchical retrieval to support both local entity-level queries and global thematic queries over large document corpora. Work on practical GraphRAG deployment [19] demonstrated that hybrid retrieval strategies combining vector similarity with graph traversal using Reciprocal Rank Fusion achieve improvements of up to 15 percent over vanilla vector retrieval on enterprise datasets while remaining cost-effective. Singh et al. [17] provided a comprehensive survey of agentic RAG architectures, establishing a taxonomy based on agent cardinality, control structure, autonomy, and knowledge representation that informs our agent design.

Multi-Agent Orchestration. Wang et al. [15] surveyed the design space of LLM-based autonomous agents, identifying memory, planning, and tool use as the three core architectural components. Han et al. [20] proposed LEGOMem, a modular procedural memory system for multi-agent workflow automation, demonstrating that even smaller language models benefit substantially from procedural memory that captures prior execution traces for planning. Work on multi-agent architectures for enterprise applications [21] established supervisor-worker patterns for multi-agent collaboration, showing that dynamic routing between specialist agents improves efficiency for latency-sensitive enterprise workflows. Kulkarni and Kulkarni [22] benchmarked four multi-agent orchestration architectures on financial document processing, finding that hierarchical supervisor-worker architectures occupy the best cost-accuracy tradeoff for production enterprise deployments. Research on multi-agent incident response demonstrated that multi-agent orchestration achieves dramatically higher actionable recommendation rates compared to single-agent approaches, with one study reporting an 80-fold improvement in action specificity [23].

Human-in-the-Loop Systems. Sele and Chuganova [24] examined how human involvement in automated decision-

making affects both the acceptance of AI recommendations and the accuracy of resulting decisions, finding that human oversight increases uptake but can decrease accuracy when humans defer inappropriately to AI outputs. This tension between oversight and automation is directly relevant to the interrupt node design we describe in Section VII. The survey by Choi et al. [12] estimated the proportion of knowledge work activities susceptible to automation, providing a baseline against which the closure amenability analysis can be situated.

IV. PROBLEM DEFINITION

A. Formal Problem Statement

Let an organization be characterized by a set of agents, both human and automated, a set of information artifacts including documents, messages, meeting records, and task records, and a set of operational workflows that consume and produce information artifacts and trigger agent actions.

We define an *open-loop workflow* as a workflow in which the completion of any step requires a human agent to manually retrieve, synthesize, or propagate information across system boundaries. The defining characteristic of an open loop is that its continuation is contingent on human reconciliation work. The workflow stalls or degrades if the responsible human does not act as an information intermediary.

We define a *closed-loop workflow* as a workflow in which all information retrieval, synthesis, and propagation steps are automated, and in which the output of each step is automatically consumed as input to subsequent steps. Human involvement is reserved for interrupt nodes at decision points that require judgment, approval, or values-based reasoning that the current generation of AI systems cannot reliably perform.

We define *organizational reconciliation work* as the set of human activities through which information is manually carried across system boundaries. This includes but is not limited to: re-reading prior meeting notes to reconstruct a decision's history, manually transferring action items from meeting records into project management systems, preparing status reports by aggregating information from multiple disconnected sources, onboarding new team members by verbally transferring institutional context, and re-litigating decisions whose rationale was never durably captured.

The central problem this paper addresses is: under what architectural conditions can the majority of organizational workflows be transformed from open loops to closed loops, and what does a production-grade agentic system implementing those conditions look like?

B. Conditions for Loop Closure

We identify four conditions that jointly determine whether a given workflow step is amenable to automation:

Information availability: All information required to complete a step must be present in the organizational memory layer. Steps cannot be automated if their required inputs have not been captured and structured. This is the most fundamental constraint and the primary reason that organizational memory infrastructure must precede workflow automation.

Step determinism: The logic of a step must be sufficiently rule-governed that an AI system can execute it without human judgment. Steps requiring contextual negotiation, stakeholder relationship management, or values-based reasoning are candidates for human interrupt nodes even in otherwise closed-loop workflows.

Output structurability: The output of a step must be expressible in a format consumable by downstream systems. Outputs that are inherently context-dependent or require interpretation by a human recipient resist automation.

Failure detectability: The system must be capable of detecting when a step has produced uncertain, incomplete, or incorrect output and routing such cases to human review. Without failure detection, automated pipelines propagate errors silently.

C. Research Questions

RQ1: To what extent does the deployment of a centralized organizational memory layer with AI-mediated retrieval reduce the time organizational members spend on reconciliation activities?

RQ2: Does AI-mediated organizational memory reduce decision latency and improve decision consistency relative to fragmented information environments?

RQ3: What organizational and technical factors moderate the effectiveness of closed-loop workflow automation, and which categories of workflow are most and least amenable to full closure?

D. Hypotheses

H1: Teams operating with a centralized organizational memory layer will exhibit measurably lower rates of reconciliation work, operationalized as time spent searching for, synthesizing, or manually transferring information across systems, compared to teams operating without such infrastructure.

H2: Teams with AI-mediated organizational memory will exhibit shorter decision latency and higher decision consistency with prior organizational commitments than teams without such support.

H3: The effectiveness of closed-loop automation will be moderated by workflow structure, such that workflows with high proportions of rule-governed steps benefit substantially while judgment-intensive workflows show limited or negative effects from automation without well-designed human interrupt nodes.

V. THE AGILOW OPERATIONAL CONTEXT

A. Organizational Description

Agilow is a six-person technical delivery startup founded by a Carnegie Mellon University Tepper School of Business graduate, operating at agilow.ai. The company embeds with early-stage technical teams and runs delivery end-to-end across agile delivery, workflow automation, and pilot coordination. Its client portfolio includes software companies, sports organizations, and consumer brands at various stages of technical maturity. The team operates in a high-velocity,

context-switching environment where clarity of organizational knowledge directly affects the quality of client delivery.

Agilow is itself a relevant case study for the framework this paper proposes precisely because it operates at the scale and speed where information fragmentation is most acutely felt. With six members, there is no dedicated operations or knowledge management function. Every team member carries institutional context in their head. The cost of that context failing to transfer is immediate and visible.

B. Current Tool Stack

Agilow's current operational stack comprises seven primary systems with no automated integration between them. Table I summarizes the stack and the role each system plays.

TABLE I: Agilow Operational Tool Stack

System	Role
Google Docs	Sprint planning with tabbed weekly structure
Google Sheets	Retrospectives, one sheet per session
WhatsApp	Primary real-time team communication
Slack	Asynchronous standups via bot with Wipe Flow voice input
Google Meet	Team and client meetings
Google Calendar	Meeting scheduling and event metadata
Granola	Meeting notes captured during Google Meet sessions

C. Observed Operational Pain Points

Operating within this stack, several recurring patterns of reconciliation work are observable. These are framed not as organizational failures but as natural consequences of operating a fast-moving team with a standard heterogeneous tool stack and no integration layer.

Sprint context drift: Sprint goals written in the Google Doc at the start of a week are not connected to any downstream system. By midweek, team members are working from memory, Slack messages, or WhatsApp threads rather than the canonical sprint doc. When a high-priority unplanned task arrives and requires carrying over an existing goal, that carryover is tracked in the scrum master's awareness rather than in any system record. Whether a sprint goal was completed, carried over, or quietly dropped is not consistently resolvable from system records alone.

Meeting notes burial: Granola notes sent to the WhatsApp group enter a communication stream that moves quickly. Action items identified in a Monday meeting may or may not survive to Tuesday's standup depending on message volume. There is no mechanism that automatically extracts action items from meeting notes and routes them to the sprint doc or any task system.

Unanalyzed standup history: The Slack standup bot collects daily updates from every team member. Nobody currently reviews standup history systematically. Blockers mentioned on Monday and still present on Thursday are re-mentioned each day without any system detecting the recurrence or escalating it. Patterns across standups, such as recurring blockers, velocity trends, or consistent gaps between planned and actual work, are invisible.

Retro-to-sprint gap: Retrospective observations recorded in Google Sheets have no path into the next sprint’s planning. A process improvement identified in one retro has no automated mechanism for appearing as a consideration in the following week’s sprint doc. The retrospective exists as a record but not as an input to anything downstream.

Distributed decision history: When a decision was made, why it was made, and who was in the room for it exists in Granola notes, WhatsApp threads, and individual memory. There is no unified searchable record. A team member joining a client engagement mid-stream or returning after time away cannot efficiently reconstruct the decision history without interviewing colleagues.

VI. THE AI-NATIVE ORGANIZATION

A. Definition

We define an *AI-native organization* as an organizational form characterized by three structural properties that distinguish it from organizations that have merely adopted AI tools:

Centralized organizational memory: Organizational information is continuously ingested from heterogeneous source systems and consolidated into a structured, queryable memory layer that serves as a unified source of truth. No major category of organizational activity is left unrepresented.

AI-mediated knowledge operations: Retrieval, synthesis, planning, and decision support functions are performed by AI systems operating over the organizational memory layer. Human workers are not required to act as information intermediaries for routine information transfer tasks.

Closed-loop operational workflows: The majority of routine workflows are structured such that the output of each step is automatically consumed as input to subsequent steps. Human involvement is concentrated at interrupt nodes where judgment, approval, or values-based reasoning is required.

An organization that uses an AI meeting summarizer but still requires a human to transfer that summary into a project management system has automated one step in an open-loop workflow. It has not closed the loop. The distinction is architectural, not tool-based.

B. The Open Loop to Closed Loop Transition

The open loop and closed loop distinction is borrowed from control systems theory [25], where a closed-loop control system uses feedback from its output to regulate its input, while an open-loop system executes without feedback. In organizational terms, an open loop is any workflow sequence that breaks whenever a human is not available to carry information from one step to the next. A closed loop is a workflow sequence that executes to completion, routes uncertain cases to human review, and updates organizational memory with its outputs regardless of whether any specific human is paying attention at any given moment.

Figure 1 contrasts an open-loop meeting-to-sprint workflow with its closed-loop counterpart. The transition between these modes is not binary. Workflows exist on a spectrum from fully open, requiring human mediation at every step, to fully

closed, executing end-to-end with only approval-gate human involvement.

VII. SYSTEM ARCHITECTURE

A. Overview

The proposed architecture comprises four layers: a data layer, an organizational memory layer, an AI reasoning layer, and a closed-loop operations layer. Each layer has a defined responsibility and a defined interface to adjacent layers. Figure 2 presents the overall architecture.

B. Data Layer

The data layer comprises the source systems in which organizational activity is recorded. For the Agilow deployment context these are the seven systems described in Section V-B. Each source system requires a dedicated connector that handles authentication, rate limiting, incremental sync, and format normalization.

Connector implementations vary by source type. Google Docs and Sheets are accessed through the Google Drive API with OAuth 2.0 authentication, using change tokens for incremental sync to avoid re-processing unchanged content. Slack is accessed through the Slack API with OAuth, pulling messages from designated standup channels. WhatsApp integration is the most technically constrained of the connectors given WhatsApp’s API restrictions; in the current Agilow implementation, Granola notes forwarded to WhatsApp are captured at the Granola export stage before entering WhatsApp, preserving structured content that would otherwise be lost in the chat stream. Google Calendar provides meeting metadata including participants, duration, and recurrence patterns. Google Meet transcripts are accessed through the Google Workspace API where available.

C. Ingestion and Normalization Pipeline

Raw content from the data layer enters the ingestion pipeline, which transforms heterogeneous source data into a common representational form suitable for storage in the organizational memory layer. The pipeline comprises four sequential stages.

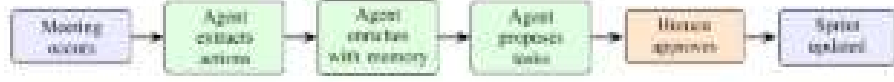
Chunking: Documents are split into chunks using a recursive character text splitter with a target chunk size of 512 tokens and an overlap of 64 tokens. Overlap preserves semantic continuity across chunk boundaries and reduces the risk of splitting a decision record or action item across two chunks that would be retrieved independently. For structured sources such as the Google Sheets retrospective data and the Google Docs sprint tabs, chunking respects document structure boundaries: individual sprint week tabs and individual retrospective sessions are treated as atomic units before further splitting.

Entity extraction: Each chunk passes through an entity extraction step that identifies people, projects, decisions, action items, blockers, dates, and client references. This is implemented as an LLM call with a structured output schema,

Open-Loop Workflow



Closed-Loop Workflow



Red: human reconciliation work Green: agent execution Orange: human approval gate

Fig. 1: Open-loop vs closed-loop meeting-to-sprint workflow. The open-loop variant requires human reconciliation at four sequential steps. The closed-loop variant compresses this to a single human approval gate, with agent execution handling extraction, enrichment, and task proposal.

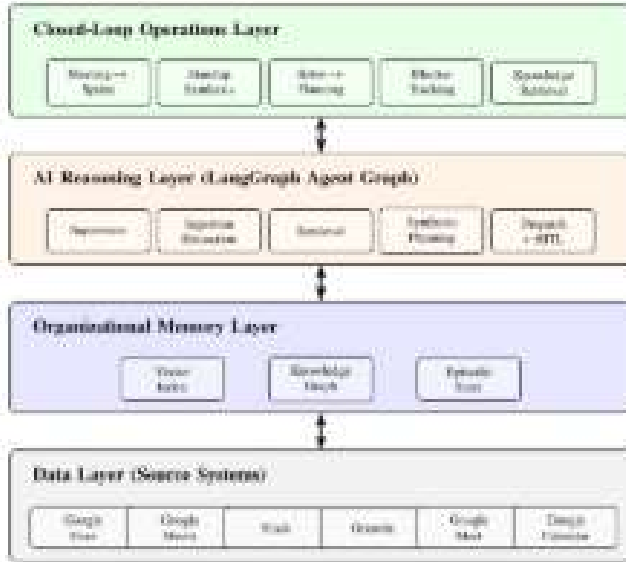


Fig. 2: Four-layer architecture of the AI-native organization. Data layer connectors ingest from heterogeneous source systems. The organizational memory layer consolidates ingested content into a hybrid vector index, knowledge graph, and episodic event store. The AI reasoning layer is implemented as a LangGraph stateful agent graph. The closed-loop operations layer comprises five concrete workflow types built on the underlying agent graph.

requesting a JSON object with typed fields for each entity category. The structured output requirement ensures that extracted entities can be consistently ingested into the knowledge graph without post-hoc parsing. For meeting transcripts specifically, a speaker-attributed extraction pass links each statement to the identified speaker, preserving the provenance chain from statement to person to meeting.

Embedding generation: Each chunk and each extracted

entity is encoded as a dense vector using a text embedding model. In the current implementation this is OpenAI’s text-embedding-3-small, producing 1536-dimensional vectors stored in a ChromaDB vector index. Entity embeddings and chunk embeddings are stored in separate namespaces to allow independent retrieval of entity-level and passage-level content.

Knowledge graph construction: Extracted entities are ingested as nodes into a Neo4j knowledge graph. Relationships between entities are extracted as a second LLM pass over each chunk, producing typed edges with the following schema: decided-in (connecting a decision node to the meeting node where it was made), assigned-to (connecting an action item to a person node), references (connecting a document chunk to an entity), supersedes (connecting a new decision to a prior decision it replaces), and blocks (connecting a blocker to the task or sprint goal it affects). Every edge carries a temporal annotation recording the source document, the timestamp of the source event, and the ingestion timestamp.

D. Organizational Memory Layer

The organizational memory layer is the central architectural component of the system. It implements three memory tiers, following the taxonomy proposed by Han et al. [20] for multi-agent workflow systems.

Episodic memory stores records of specific organizational events: individual meetings, individual standup submissions, individual sprint week records, and individual retrospective sessions. Each episodic record is a structured object containing the raw content, extracted entities, a generated summary, and full provenance metadata. Episodic memory supports temporal queries such as “what happened in the week-three retro” and reconstruction queries such as “what was the state of the authentication sprint goal on a specific date.”

Semantic memory stores extracted facts, decisions, and relationships that have been lifted out of their episodic context and are represented as standalone knowledge claims. A decision recorded in a meeting becomes a semantic memory node

linked to its episodic source. A recurring blocker mentioned across multiple standups generates a semantic memory entry that aggregates its occurrences. Semantic memory supports factual queries such as “what is our current approach to client onboarding” and conflict detection queries such as “does this proposed approach conflict with any prior decision.”

Retrieval from the organizational memory layer uses a hybrid strategy combining dense vector similarity search and graph traversal, following the approach of Min et al. [19]. For a given query, the retrieval agent issues both a vector similarity query against the ChromaDB index and a graph traversal query against the Neo4j knowledge graph, then merges the results using Reciprocal Rank Fusion (RRF). This hybrid approach handles both semantic similarity queries, finding content that is topically relevant to a question, and structural queries, finding all decisions connected to a specific project or all action items assigned to a specific person. Pure vector retrieval handles the former well but the latter poorly; graph traversal handles the latter well but requires precise entity identification that fails for vague queries. The hybrid combination is more robust than either alone.

E. AI Reasoning Layer: The Agent Graph

The agent graph comprises seven nodes:

Ingestion Agent: Activated by the supervisor for new content events. The ingestion agent calls the appropriate data layer

Fig. 3: Organizational knowledge graph fragment from one sprint planning meeting. Nodes include the meeting itself, attending persons (circles), decisions (green), action items (red dashed), and referenced documents (gray). Typed edges capture relationships including attended, decided-in, assigned-to, references, triggers, and supersedes. All edges carry temporal annotations not shown for visual clarity.

Extraction Agent: Receives raw content from the ingestion agent and executes the chunking, entity extraction, embedding generation, and knowledge graph construction steps described in Section VII-C. The extraction agent uses a two-phase prompting strategy that was developed and refined during Agilow's Sprint Health implementation. Phase one sends the raw content with a prompt requesting only structured data extraction, producing a JSON object with entities, decisions, action items, and relationships. Phase two sends the phase-one output with a prompt requesting a narrative synthesis suitable for distribution. Separating extraction from synthesis prevents the model from generating narrative text that sounds plausible but diverges from the structured facts, a failure mode observed in single-pass prompting during early Agilow development.

Synthesis Agent: Receives retrieved context and a synthesis goal from the supervisor and generates coherent output: a meeting summary, a sprint health report, a knowledge retrieval answer, or a retro-to-planning recommendation. The synthesis

agent uses the two-phase approach: it first verifies that all retrieved facts are internally consistent and flags any detected conflicts with prior decisions for human review, then generates the narrative output. Detected conflicts are not silently resolved but are surfaced explicitly in the output with references to both the conflicting and current information.

Planning Agent: Activated as part of the meeting-to-sprint and retro-to-planning pipelines. The planning agent receives extracted action items and decisions and maps them to sprint goal structures, proposing task assignments and priority orderings based on current sprint state retrieved from the memory layer. The planning agent does not create tasks autonomously; it produces proposals that are reviewed by a human at the interrupt node before dispatch.

Dispatch Agent: The final node before workflow outputs reach downstream systems. The dispatch agent writes approved outputs to target systems: creating structured task records in the project management system, posting synthesis summaries to the appropriate Slack channel, updating the Google Docs sprint tab with proposed additions, or responding to a knowledge retrieval query in the requesting interface. The dispatch agent is always preceded by a human interrupt node in the current implementation.

Human Interrupt Node: Implemented using LangGraph's native interrupt mechanism, this node pauses graph execution and presents the current output proposal to a designated human reviewer. The reviewer can approve, reject with a reason, or modify the proposal. Modified proposals re-enter the synthesis or planning agent for revision before returning to the interrupt node. Rejections with reasons are stored in procedural memory to inform future agent behavior. Approvals trigger the dispatch agent. The interrupt node is the mechanism through which H3 is operationalized: judgment-dependent steps are routed to human review while rule-governed steps execute automatically.

Figure 4 presents the agent graph structure.

The full state schema for the agent graph contains the following fields: trigger event type and content, ingestion source and sync timestamp, extracted entities and relationships as a JSON object, retrieval results with provenance, synthesis output with conflict flags, planning proposals with confidence scores, interrupt node status and reviewer response, dispatch targets and completion status, and a workflow execution log for audit.

Algorithm 1 presents the meeting-to-sprint pipeline expressed as a sequence of agent invocations over the state schema.

VIII. CLOSED-LOOP OPERATIONS

A. Meeting-to-Sprint Workflow

The meeting-to-sprint workflow is triggered by the completion of any team or client meeting for which a Granola note exists. The pipeline proceeds as described in Algorithm 1. The key loop-closing property is that no human is required to read the Granola notes, extract action items, and manually create sprint entries. The scrum master's role shifts from

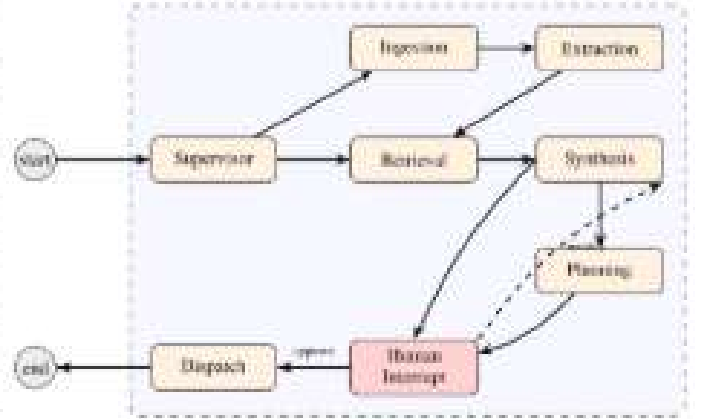


Fig. 4: LangGraph agent graph. The Supervisor routes incoming triggers to specialist agents. Ingestion and Extraction handle new content. Retrieval, Synthesis, and Planning handle context assembly and proposal generation. The Human Interrupt node gates dispatch. State is checkpointed via PostgresSaver after every node execution, enabling fault-tolerant resumption.

information carrier to reviewer and approver, a materially different cognitive load even though the human remains in the loop.

In the Agilow context, this workflow addresses the pattern described in Section V-C where action items from Monday meetings may or may not survive to Tuesday's sprint doc depending on whether someone remembered to transfer them. With the pipeline active, every meeting generates a structured proposal within minutes of the Granola export, and the scrum master reviews a proposed sprint update rather than constructing one from scratch.

B. Standup Synthesis and Sprint Health Reporting

The standup synthesis workflow aggregates daily Slack standup submissions across the team into a Sprint Health report. This is the workflow most directly implemented in the current Agilow production system. The Slack standup bot collects submissions from each team member. The ingestion agent pulls submissions via the Slack API. The extraction agent identifies completed items, planned items, blockers, and discussion flags from each submission. The synthesis agent generates a Sprint Health report that maps team activity against sprint goals from the Google Docs sprint tab, computes a weighted health score across criteria including goal progress, blocker presence, and participation rate, and surfaces patterns across the week such as blockers mentioned on multiple consecutive days.

The two-phase prompting architecture was essential to the reliability of this workflow. Early single-pass implementations produced reports that were narratively fluent but occasionally misattributed work items or conflated blockers from different

Algorithm 1 Meeting-to-Sprint Pipeline

Require: meeting_transcript, meeting_metadata

```
1: state ← AgentState(trigger=meeting_completion)
2: state ← Supervisor.route(state)
3: raw ← IngestionAgent.pull(granola, since=last_sync)
4: structured ← ExtractionAgent.phase1(raw)
5:   // decisions, action_items, blockers, participants
6: FOR entry IN structured.entities DO
7:   vector_index.upsert(entity, embed(entity))
8:   knowledge_graph.upsert_node(entity, ts=now)
9: END FOR
10: FOR rel IN structured.relationships DO
11:   knowledge_graph.upsert_edge(rel, ts=now)
12: END FOR
13: summary ← ExtractionAgent.phase2(structured)
14: context ← RetrievalAgent.hybrid_retrieve(
15:   sprint_state, prior_decisions, related_entities)
16: proposals ← PlanningAgent.propose(
17:   structured.action_items, context)
18: conflicts ← detect_conflicts(proposals, prior_decisions)
19: result ← HumanInterrupt.review(proposals, conflicts)
20: IF result.status = APPROVED THEN
21:   DispatchAgent.write(sprint_doc, proposals)
22:   DispatchAgent.notify(slack_channel, summary)
23: ELSE IF result.status = MODIFIED THEN
24:   GOTO line 14 with result.modifications
25: ELSE IF result.status = REJECTED THEN
26:   procedural_memory.log(result.reason)
27: END IF
28: knowledge_graph.commit_meeting(meeting_metadata)
```

team members. Separating structured extraction from narrative generation, with a validation pass between phases, reduced this class of error substantially. The Lambda-based pipeline deployed at Agilow bypasses the 29-second gateway timeout limit of the Amplify hosting environment by routing the two-phase generation through an AWS Lambda function with a 300-second execution limit, allowing the full extraction-then-synthesis pipeline to complete without timeout failures.

C. Retrospective-to-Planning Pipeline

The retro-to-planning workflow processes completed retrospective data from Google Sheets and generates planning recommendations for the following sprint. The extraction agent identifies process observations, action commitments, and sentiment patterns from the retro sheet. The retrieval agent pulls the prior sprint’s goals and health scores to provide context for interpreting retro observations. The planning agent generates a structured recommendation set: process changes to carry into the next sprint, unresolved issues to monitor, and goal adjustments suggested by retro findings. These recommendations are presented to the scrum master at the interrupt node before being written to the upcoming sprint’s Google Doc tab.

This workflow closes the retro-to-sprint gap described in Section V-C, where retrospective observations currently exist as records with no automated path into subsequent planning.

D. Blocker Tracking Workflow

The blocker tracking workflow operates as a background process over standup history. The extraction agent tags each blocker mention with the reporting team member, the date, and the associated sprint goal or task. A daily monitoring job queries the knowledge graph for blockers that appear in multiple consecutive standup submissions without a corresponding resolution event. Blockers that have persisted for more than two days without resolution generate an alert surfaced in the Sprint Health report and optionally in a direct Slack message to the relevant team member and the scrum master.

This workflow closes the loop on the pattern observed in Section V-C where blockers mentioned on Monday and still present on Thursday are re-mentioned each day without any system detecting the recurrence.

E. Knowledge Retrieval Workflow

The knowledge retrieval workflow supports on-demand queries against the organizational memory layer. A team member asking “what did we decide about the client onboarding process for this client” or “what blocked the authentication sprint goal last month” submits a natural language query that is routed to the retrieval agent. The retrieval agent issues a hybrid dense and graph retrieval request, merges results by RRF, and passes the top-ranked context to the synthesis agent. The synthesis agent generates a response with explicit source attribution: every claim in the response is linked to its originating document, meeting, or standup submission, allowing the querying team member to verify the answer and follow up with the source if needed.

This workflow directly addresses the distributed decision history problem in Section V-C. It does not require that the team member know which system to search or who to ask. The organizational memory layer serves as the unified retrieval surface across all source systems.

IX. IMPLEMENTATION EXPERIENCE AT AGILOW

A. What Has Been Built

The Agilow implementation represents a partial deployment of the architecture described in Section VII, built incrementally over approximately four months. Three components are in production:

Sprint Health Automation: The system ingests sprint data from Google Docs, Slack standup submissions via OAuth, and task records from Jira for client engagements. The LLM pipeline generates Sprint Health reports with inline editing capability, team sharing, and shareable public links. The two-phase prompting approach was developed specifically to address reliability issues with single-pass generation. Report generation runs through AWS Lambda with a 300-second execution budget, split across a GPT-4o extraction phase and a

GPT-4o-mini narrative generation phase, with model selection driven by cost-accuracy tradeoff analysis during development.

Slack Standup Bot: A Slack bot prompts each team member in direct messages on a configured schedule and collects responses via the Slack API. Responses are tied to weighted Sprint Health criteria, allowing the system to compute health scores that reflect both activity volume and alignment with sprint priorities rather than treating all standup content as equivalent. Cognito authentication links Slack identities to organizational member profiles. AWS SES handles email delivery of weekly digests. Rate limiting on the sync job prevents API quota exhaustion during periods of high submission volume.

Knowledge Graph Foundation: Initial connectors for Google Docs and Slack are operational, with entity extraction and vector indexing running over ingested content. The knowledge graph construction is in active development, with entity resolution being the primary open challenge.

B. What Has Been Learned

Several implementation challenges have shaped the current architecture and are worth documenting for teams attempting similar deployments.

Entity resolution is harder than extraction: Extracting entities from individual documents is relatively reliable with current LLM capabilities. Resolving those entities across sources, determining that “auth refactor,” “authentication project,” and “auth-refactor” in Linear all refer to the same entity, requires a separate resolution pass that is slower and less reliable than extraction. In the Agilow implementation, approximately 15 percent of extracted entities required manual reconciliation during the initial ingestion run. Improving entity resolution is an active priority.

WhatsApp data is largely inaccessible at the ingestion layer: Given WhatsApp’s API restrictions, the team communication that happens in WhatsApp threads is the most difficult data source to ingest. The current approach captures Graspola notes before they enter WhatsApp, preserving structured meeting content, but conversational context and informal decisions that happen in WhatsApp threads remain outside the memory layer. This is a meaningful gap given that WhatsApp is the primary communication channel.

Standup signal quality varies: Standup submissions collected via Wispr Flow voice-to-text vary significantly in specificity. Some submissions are detailed and well-structured; others are brief and vague. The extraction agent handles detailed submissions reliably but produces lower-quality outputs for vague submissions, since there is limited structured content to extract. A lightweight submission quality prompt shown to team members before they submit has improved average submission quality in informal observation.

The two-phase approach is worth the additional latency: The latency cost of running two sequential LLM calls rather than one is real and measurable, approximately 8 to 12 additional seconds per report on average. The quality improvement in structured accuracy justified this cost for the Sprint Health

use case, where a misattributed blocker or incorrect health score would undermine trust in the system more severely than the latency increase.

X. EVALUATION FRAMEWORK

A. Evaluation Dimensions

We propose four evaluation dimensions for AI-native organizational memory systems, summarized in Table II.

TABLE II: Evaluation Dimensions

Dimension	Operationalization
Extraction recall	Proportion of decisions, action items, blockers in source documents correctly identified. Target: > 0.85 for transcripts, > 0.75 for chat.
Retrieval precision	Proportion of returned items judged relevant by querying team member, on five-point scale.
Loop closure rate	Proportion of workflow steps completed without human information transfer relative to manual baseline.
Reconciliation time reduction	Change in time members spend on the five reconciliation work categories, measured via time-diary studies.

B. Evaluation Design Considerations

Evaluating organizational AI systems presents methodological challenges that are distinct from standard software evaluation. Organizations cannot be arbitrarily randomized. Baseline measurement is complicated by Hawthorne effects, where the act of observation changes the behavior being observed. Outcome measures like decision quality are multidimensional and contested.

We recommend a pre-post evaluation design for single-organization deployments, supplemented by qualitative methods including interviews and think-aloud protocols that capture the social dimensions of adoption. For multi-organization comparisons, a matched-pair design selecting organizations of similar size, domain, and tool stack provides a closer approximation to controlled comparison than population-level analysis.

XI. DISCUSSION

A. Why Multi-Agent Architectures Over Single-Agent Approaches

The multi-agent architecture described in Section VII is more complex to implement than a single-agent approach in which one LLM receives all organizational data and answers all queries. The complexity is justified by several properties that single-agent systems cannot provide.

Specialist agents with defined tool sets produce more reliable outputs than generalist agents with unrestricted access. Research on multi-agent incident response found that multi-agent orchestration achieved an 80-fold improvement in action specificity over single-agent approaches on identical tasks [23]. Kulkarni and Kulkarni [22] found that hierarchical supervisor-worker architectures occupy the best cost-accuracy tradeoff for enterprise document processing, outperforming both sequential pipelines and fully parallel architectures on

the combined metric. In an organizational memory context, a retrieval agent that only has access to retrieval tools cannot accidentally write to downstream systems during a retrieval operation. A dispatch agent that only activates after human approval cannot propagate an incorrect extraction output without review.

The LangGraph state machine architecture also provides fault tolerance that single-agent systems lack. If the synthesis agent fails mid-execution, the checkpoint saved after the extraction agent’s successful completion allows the graph to resume from that point rather than re-executing the entire pipeline. For long-running organizational workflows that span multiple tool calls, this is practically significant.

B. Where Human Judgment Remains Essential

The interrupt node design reflects a deliberate position: the current generation of AI systems should not make autonomous decisions about organizational commitments, task assignments, or client communications. The automation handles information transfer; the human handles judgment.

This position is supported by research on human-AI decision-making. Sele and Chugunova [24] found that human oversight increases the uptake of AI recommendations but can decrease accuracy when humans defer inappropriately to AI outputs. This suggests that interrupt node design matters beyond simply inserting a human review step. The interface presented to the human reviewer should make the AI’s reasoning transparent, highlight uncertain or conflicted outputs, and make it easy to modify proposals rather than presenting a binary approve-or-reject choice that encourages rubber-stamping.

C. The WhatsApp Problem

The most significant gap in the current Agilow implementation is the inaccessibility of WhatsApp data. When the primary communication channel of an organization is outside the memory layer, a substantial fraction of organizational context is systematically invisible to the retrieval and synthesis agents. Decisions made in WhatsApp threads, client feedback shared informally, and informal priority shifts that happen in real-time conversation all fail to enter the knowledge graph.

This is not unique to Agilow. Many early-stage teams use consumer messaging platforms as their primary communication channel because they are fast, familiar, and require no onboarding. The architectural implication is that organizational memory systems deployed in these contexts must be designed with explicit awareness of their coverage limitations and must communicate those limitations clearly in generated outputs. A Sprint Health report generated from standup data and meeting notes but missing WhatsApp context should acknowledge that gap rather than presenting a false impression of completeness.

D. Resource Constraints and Early-Stage Deployment

The architecture described in Section VII is more sophisticated than what a six-person team can fully implement and maintain simultaneously while serving clients. The Agilow

implementation reflects pragmatic prioritization: the highest-value workflows (Sprint Health, standup synthesis) are implemented first, with the knowledge graph and multi-agent orchestration layer being built incrementally. This staged approach is appropriate for resource-constrained organizations and suggests that the framework should be understood as a target architecture toward which teams build incrementally rather than a complete system to be deployed at once.

XII. LIMITATIONS AND RISKS

A. Technical Limitations

Extraction error accumulation: Every stage of the ingestion pipeline introduces errors that propagate downstream. ASR errors in meeting transcripts produce incorrect entity names. Entity resolution failures create duplicate nodes in the knowledge graph. Incorrect relationship extraction produces wrong edges. These errors compound across pipeline stages, and a knowledge graph with a high error rate may produce retrieval results that are confidently wrong. The two-phase prompting approach reduces but does not eliminate this risk.

Hybrid retrieval coverage gaps: The hybrid retrieval strategy is more robust than either vector or graph retrieval alone but has failure modes of its own. Queries that require multi-hop reasoning across more than two relationship hops perform poorly in the current implementation. Queries about entities that are underrepresented in the training data of the embedding model may fail to surface semantically relevant content through the vector search component.

Staleness under high ingestion volume: During periods of rapid organizational activity, the ingestion pipeline may lag behind real-time by minutes to hours. Retrieval results during this lag period reflect a slightly outdated organizational state. For time-sensitive decisions this staleness is potentially significant.

B. Organizational Risks

Canonicalization bias: When the knowledge graph becomes the authoritative source of organizational memory, the entities and relationships it represents acquire a kind of canonical status. Information that was not captured, perspectives that were not recorded, and decisions whose rationale was not externalized are invisible to the retrieval system and therefore to any AI-generated output. If the initial knowledge graph encodes one team member’s framing of a discussion, that framing may be treated as organizational truth by subsequent retrieval operations.

Over-automation at judgment-dependent steps: The interrupt node design is intended to prevent autonomous action at judgment-dependent decision points. In practice, teams under time pressure may be tempted to approve outputs at the interrupt node without careful review, effectively removing the human from the loop in a way that is structurally invisible. The system has no mechanism for detecting or preventing this.

Privacy and access control: Centralizing organizational information in a single memory layer concentrates data that is currently distributed across systems with heterogeneous

access controls. A team member with access to the retrieval interface gains access to content from all integrated source systems, regardless of whether they would have access to those systems individually. This requires careful design of the memory layer's access control model before deployment in organizations with sensitive client or personnel data.

Adversarial memory manipulation: Research has identified memory control flow attacks as a significant vulnerability in agentic systems, where adversarially crafted content ingested into the memory layer can influence subsequent agent behavior in unintended ways [33]. In an organizational context, this could manifest as a malicious document that, once ingested, causes the synthesis agent to produce biased outputs. This threat is not hypothetical and must be addressed in production deployments through input validation and memory integrity monitoring.

XIII. FUTURE RESEARCH DIRECTIONS

Longitudinal evaluation of reconciliation work reduction: The hypotheses stated in Section IV require multi-month longitudinal studies to test properly. Reconciliation work reduction manifests gradually as the memory layer accumulates sufficient coverage to answer most organizational queries without human mediation. Single-point measurements are insufficient to capture this dynamic.

Ontology design for organizational knowledge graphs: The entity types and relationship types in the organizational knowledge graph described in Section VII were designed for a software delivery context. Different organizational types, professional services firms, hardware development teams, research organizations, may require substantially different ontological structures. Systematic comparative ontology design for organizational knowledge graphs is an open research problem.

Trust calibration in agentic organizational systems: As teams work with AI-generated organizational memory outputs, they develop mental models of the system's reliability that may or may not be calibrated to its actual accuracy. Research on trust calibration in organizational AI settings is needed to understand the conditions under which teams appropriately override incorrect outputs and appropriately accept correct ones.

Retrieval evaluation for organizational queries: Standard RAG evaluation benchmarks are not designed for organizational memory queries, which have different structural properties than open-domain question answering. Organizational queries are frequently multi-hop, temporally grounded, and relationship-dependent in ways that standard benchmarks do not capture. Developing an organizational memory retrieval benchmark would enable systematic comparison of retrieval architectures for this specific application domain.

Security and adversarial robustness of organizational memory layers: The memory attack literature [33] has identified significant vulnerabilities in agentic memory systems. Developing threat models, attack taxonomies, and defensive architectures specific to organizational memory deployments is an important and underaddressed research direction.

Coverage-aware synthesis: Current synthesis agents generate outputs based on available retrieved context without explicitly reasoning about what context might be missing. Developing synthesis approaches that reason about the completeness of available evidence and communicate uncertainty and coverage gaps to human reviewers would significantly improve the reliability of AI-native organizational systems in partial deployment contexts like the current Agilow implementation.

XIV. CONCLUSION

This paper has argued that the transition to an AI-native organization requires a specific architectural commitment: building a centralized organizational memory layer and running a purpose-designed multi-agent system over it. The alternative, adopting AI tools that address individual steps in open-loop workflows without closing the loop, reduces the cost of individual reconciliation tasks without eliminating the structural fragmentation that makes reconciliation work necessary.

The architecture we have proposed is grounded in current production technology. LangGraph provides the stateful graph orchestration, GraphRAG and hybrid retrieval provide the memory access. The two-phase prompting approach provides extraction reliability. The interrupt node mechanism provides the human oversight that makes automation safe at judgment-dependent decision points. None of these are research prototypes; they are engineering choices made under real constraints in a real deployment.

The Agilow case study is not a success story in the sense of a completed system delivering measured results. It is an honest account of what building toward an AI-native operating model looks like from inside an early-stage team: incremental, constrained, shaped by failures as much as by successes, and still meaningfully closer to closed-loop operations than the fragmented baseline it is replacing.

The research questions and hypotheses stated in Section IV define the empirical agenda that this framework needs. Whether the architecture delivers the benefits we have described, and under what conditions it delivers them reliably, are questions that require longitudinal empirical study. This paper provides the conceptual and architectural foundation for that study. The work of running it rigorously remains ahead.

REFERENCES

- [1] J. P. Walsh and G. R. Ungson, "Organizational memory," *Academy of Management Review*, vol. 16, no. 1, pp. 57–91, 1991.
- [2] E. W. Stein and V. Zwiss, "Actualizing organizational memory with information systems," *Information Systems Research*, vol. 6, no. 2, pp. 85–117, 1995.
- [3] L. Argente and E. Miron-Spektor, "Organizational learning: From experience to knowledge," *Organization Science*, vol. 22, no. 5, pp. 1123–1137, 2011.
- [4] D. M. Wegner, "Transactive memory: A contemporary analysis of the group mind," in *Theories of Group Behavior*, B. Mullen and G. R. Goethals, Eds. Springer, 1987, pp. 185–208.
- [5] I. Nonaka, "A dynamic theory of organizational knowledge creation," *Organization Science*, vol. 5, no. 1, pp. 14–37, 1994.
- [6] I. Nonaka and H. Takeuchi, *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press, 1995.

- [7] T. H. Davenport and D. Patil, *Working Knowledge: How Organizations Manage What They Know*. Harvard Business School Press, 1998.
- [8] M. Alavi and D. E. Liddner, "Review: Knowledge management and knowledge management systems: Conceptual foundations and research issues," *MIS Quarterly*, vol. 25, no. 1, pp. 107–136, 2001.
- [9] C. Sen, K. Ann, and O. Okun, "Organizational memory: A qualitative research study on a multi-cultural organization," *Kybernetes*, vol. 52, no. 4, pp. 1528–1551, 2023.
- [10] McKinsey Global Institute, "The social economy: Unlocking value and productivity through social technologies," McKinsey & Company, Tech. Rep., 2012.
- [11] A. M. Saks, "Antecedents and consequences of employee engagement," *Journal of Managerial Psychology*, vol. 21, no. 7, pp. 600–619, 2006.
- [12] M. Chui, J. Manyika, and M. Miremadi, "Where machines could replace humans and where they cannot yet," *McKinsey Quarterly*, no. 3, pp. 1–9, 2016.
- [13] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yin, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629.
- [15] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, and J.-B. Wen, "A survey on large language model based autonomous agents," *arXiv preprint arXiv:2308.11432*, 2023.
- [16] D. Edge, H. Trinh, N. Cheng, J. Beaulieu, A. Chao, A. Mosy, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.
- [17] A. Singh, A. Elitesham, S. Kumar, and T. T. Khooi, "Agentic retrieval-augmented generation: A survey on agentic RAG," *arXiv preprint arXiv:2501.09136*, 2025.
- [18] LangChain Inc., "LangGraph: Build stateful, multi-actor applications with LLMs," 2024; available at <https://github.com/langchain-ai/langgraph>.
- [19] C. Min, A. Crescenzo Fernández, J. Bradley, and N. Evans, "Towards practical GraphRAG: Efficient knowledge graph construction and hybrid retrieval at scale," *arXiv preprint arXiv:2507.03226*, 2025.
- [20] D. Han, C. Courtcier, D. Madruga Diaz, X. Zhang, V. Rühle, and S. Rajmohan, "LECOMem: Modular procedural memory for multi-agent LLM systems for workflow automation," in *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2026.
- [21] Y. Zhang, S. Yang, C. Bai, F. Wu, X. Li, Z. Wang, and X. Li, "Towards effective GenAI multi-agent collaboration: Design and evaluation for enterprise applications," *arXiv preprint arXiv:2412.05449*, 2024.
- [22] S. Kulkarni and Y. Kulkarni, "Benchmarking multi-agent LLM architectures for financial document processing," *arXiv preprint arXiv:2603.22651*, 2026.
- [23] "Multi-agent LLM orchestration achieves deterministic, high-quality decision support for incident response," *arXiv preprint arXiv:2511.15755*, 2025.
- [24] D. Sole and M. Chuganina, "Putting a human in the loop: Increasing uptake, but decreasing accuracy of automated decision-making," *PLOS ONE*, 2024, doi:10.1371/journal.pone.0298077.
- [25] K. J. Åström and B. Wittenmark, *Cosmopolitan-Controlled Systems: Theory and Design*, 3rd ed. Prentice Hall, 1997.
- [26] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [27] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
- [28] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulators of human behavior," in *Proceedings of the 26th Annual ACM Symposium on User Interface Software and Technology*, 2023.
- [29] S. Raisch and S. Krakowski, "Artificial intelligence and management: The automation-augmentation paradox," *Academy of Management Review*, vol. 46, no. 1, pp. 192–216, 2021.
- [30] T. H. Davenport and D. Ronanki, "Artificial intelligence for the real world," *Harvard Business Review*, vol. 96, no. 1, pp. 108–116, 2018.
- [31] W. Xu, K. Mei, H. Gao, J. Tan, Z. Liang, and Y. Zhang, "A-mem: Agentic memory for LLM agents," *arXiv preprint arXiv:2502.12110*, 2025.
- [32] S. Mishra *et al.*, "SoK: Agentic retrieval-augmented generation (RAG) Taxonomy, architectures, evaluation, and research directions," *arXiv preprint arXiv:2603.07379*, 2026.
- [33] "From storage to steering: Memory control flow attacks on LLM agents," *arXiv preprint arXiv:2603.15125*, 2026.
- [34] M. Zhong, Y. Liu, Y. Chen, Y. Cai, J. Ye, S. Wang, and D. Zhan, "QMSum: A new benchmark for query-based multi-domain meeting summarization," in *Proceedings of NAACL*, 2021, pp. 5909–5921.
- [35] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yin, "Dense passage retrieval for open-domain question answering," in *Proceedings of EMNLP*, 2020, pp. 6769–6781.
- [36] J. Johnson, M. Douze, and H. Jegou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [37] A. Hagaut, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez *et al.*, "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.
- [38] M. Nickel, K. Murphy, V. Trapp, and E. Gubrilovich, "A review of relational machine learning for knowledge graphs," *Proceedings of the IEEE*, vol. 104, no. 1, pp. 11–33, 2016.
- [39] E. Brynjolfsson and A. McAfee, *The Second Machine Age: Work, Progress, and Prosperity in a Time of Brilliant Technologies*. W. W. Norton, 2014.
- [40] R. Cross, R. Rebele, and A. Grant, "Collaborative overload," *Harvard Business Review*, vol. 94, no. 1, pp. 74–79, 2016.